Student Management System

Midnight Campus Full-Stack Web Application

SUBMISSION DETAILS

Prepared by	Areeha Arif
Internship Organization	Devixo Solutions
Internship Domain	Web Development
Task Number	Task 03
Technologies	React, Node.js, Express, MongoDB Atlas

01. Project Objective

The primary objective of Task 3 is to develop, test, debug, and document a robust, production-grade Student Management System (SMS). The system eliminates manual academic spreadsheets and transient mock stores by providing permanent cloud database persistence with MongoDB Atlas.

Core Project Objectives:

1. Permanent Storage: Store student records persistently in MongoDB with timestamps & validation.
2. Authentication & Security: Implement JWT authentication, hashing, and auto-session recovery.
3. Role-Based Access Control: Differentiate between Administrator (CRUD) and Student/Viewer (Read-only).
4. Search, Filters & Pagination: Deliver live multi-field search with department and semester filters.
5. Defect Resolution: Systematically diagnose and resolve all reported connection and form handling errors.

02. Technologies Used

Category Technology Version Role & Implementation Details

Frontend UI React 19.3.1 Component architecture, state hooks, and virtual DOM rendering
Build Tool Vite 8.3.4 Fast HMR dev server and client build tool with /api reverse proxy
Styling Modern CSS3 Custom Midnight Campus dark theme with Grid, Flexbox & CSS variables
Backend API Node.js / Express v24 / 5.3 RESTful API routing, JSON body parser, and CORS middleware
Database MongoDB Atlas Cloud Persistent cloud NoSQL storage for users and student documents
ODM Library Mongoose 9.11.2 Schema modeling, data validation, and database queries
Security JSON Web Token 9.0.3 Stateless Bearer token authentication and role claims
Password Hash Bcrypt.js 3.0.3 Cryptographic irreversible salt-hashing (12 rounds)

03. Website Structure

```
student-management-system/
  backend/
      middleware/ruth.js --> JWT verification & adminOnly guard
       models/Student.js --> Student schema (name, email, department, semester)
       models/User.js --> User schema (name, email, password, role)
       routes/auth.js --> /register, /login, /me endpoints
       routes/students.js --> GET, POST, PUT, DELETE with pagination & search
       .env --> PORT, MONGODB_URI, JWT_SECRET, ADMIN_SETUP_KEY
       server.js --> Express bootstrap & MongoDB Atlas connection
  frontend/
       src/services/api.js --> Centralized API client & error handling
       src/App.jsx --> Main application component & auth state
       src/App.css --> Midnight Campus dark theme & responsive rules
       vite.config.js --> Vite reverse proxy configuration
```

04. Design Approach

1. UI/UX Paradigm: The "Midnight Campus" Design System

- Theme: Deep charcoal-navy canvas (#070e1b) accented with electric lime (#a5f15b) and soft cyan.
- Layout: Sticky left sidebar navigation, top status bar, hero banner, metrics grid, and directory.
- Non-Intrusive Feedback: Replaced native alert() dialogs with animated floating toast notifications.

2. Role-Based Access Control (RBAC) Architecture

- Administrator: Onboarded using ADMIN_SETUP_KEY. Full CRUD rights (Add, Edit, Delete students).
- Student / Viewer: Read-only access to browse records, search, filter, and inspect detailed profiles.

3. Defensive Error Segregation & Data Validation

- Client-side validation: Checks name length (>=2), email regex, department, and semester (1-8).
- Centralized API layer: Catches network disconnects separately from application runtime errors.

05. Website Features

1. Authentication & Session Persistence Dual-mode card allowing users to Sign In or Register. Stored JWT tokens persist login across page refreshes via GET /api/auth/me.

2. Administrator Role Verification Prospective admins provide the system ADMIN_SETUP_KEY during registration, unlocking full database management privileges.

3. Add Student Form Validation Interactive modal validating full name (min 2 chars), valid email regex, academic department, and semester (1st-8th) with inline feedback.

4. Edit Student Profile (Null-Safe) Pre-populates existing data into form controls and dispatches PUT requests to MongoDB with immediate UI reflection.

5. Secure Record Deletion Permanent deletion guarded by confirmation prompts to prevent accidental student record loss.

6. Student Profile Details Modal Detailed profile modal displaying formatted Student ID (STU-001), MongoDB document _id, department, semester, and join timestamp.

7. Live Search & Multi-Filters Real-time client filtering across name, email, department, and ID, paired with Department and Semester dropdown selectors.

8. Synchronized Pagination Page number buttons, previous/next controls, and configurable rows-per-page selector (5, 10, 20, 50).

9. Real-Time Campus Dashboard Cards Live summary metrics for Total Students, Active Departments, and Academic Session status.

10. Animated Toast Notifications Non-blocking floating toasts (success, error, warning) that automatically dismiss after 4 seconds.

06. Code Screenshots

- **App.jsx**



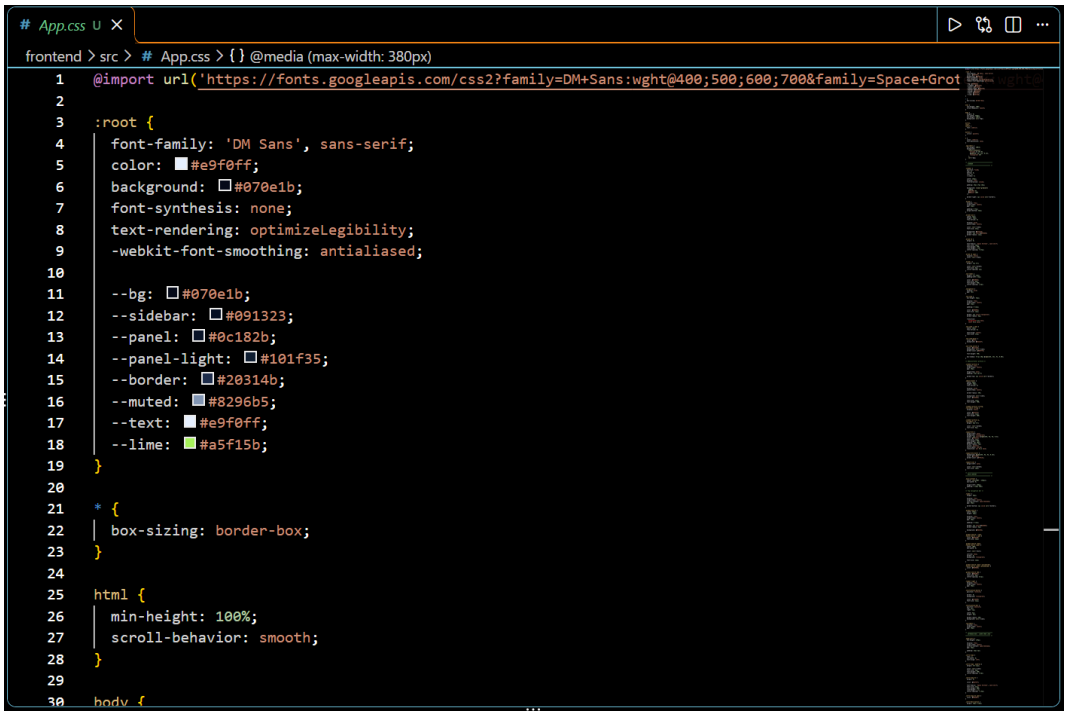
```jsx
import { useState, useEffect, useCallback } from 'react'
import api from './services/api'
import './App.css'

function formatSemester(value) {
  const number = Number(value)
  if (!number) return value || '1st'
  const suffix = number === 1 ? 'st' : number === 2 ? 'nd' : number === 3 ? 'rd' : 'th'
  return `${number}${suffix}`
}

function extractSemesterNumber(value) {
  if (!value) return '1'
  return String(value).replace(/(st|nd|rd|th)$/i, '')
}

function mapStudent(student, index = 0) {
  if (!student) return null
  const id = student._id || student.id
  const rawSemester = student.semester
  const displaySemester =
    typeof rawSemester === 'number'
      ? formatSemester(rawSemester)
      : String(rawSemester).includes('st') ||
        String(rawSemester).includes('nd') ||
        String(rawSemester).includes('rd') ||
        String(rawSemester).includes('th')
      ? rawSemester
      : formatSemester(rawSemester)
```

- **App.css**



```css
@import url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=Space+Grot

:root {
  font-family: 'DM Sans', sans-serif;
  color: #e9f0ff;
  background: #070e1b;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;

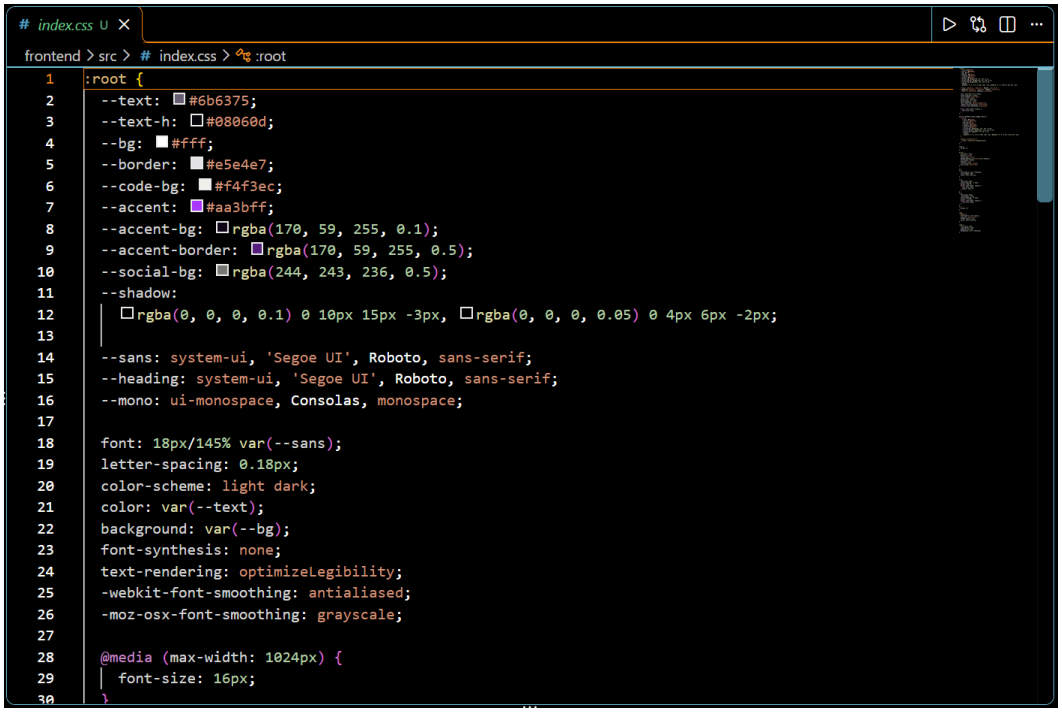
  --bg: #070e1b;
  --sidebar: #091323;
  --panel: #0c182b;
  --panel-light: #101f35;
  --border: #20314b;
  --muted: #8296b5;
  --text: #e9f0ff;
  --lime: #a5f15b;
}

* {
  box-sizing: border-box;
}

html {
  min-height: 100%;
  scroll-behavior: smooth;
}

body {
```

- **Index.css**



```
:root {
  --text: #6b6375;
  --text-h: #08060d;
  --bg: #fff;
  --border: #e5e4e7;
  --code-bg: #f4f3ec;
  --accent: #aa3bff;
  --accent-bg: rgba(170, 59, 255, 0.1);
  --accent-border: rgba(170, 59, 255, 0.5);
  --social-bg: rgba(244, 243, 236, 0.5);
  --shadow:
    rgba(0, 0, 0, 0.1) 0 10px 15px -3px, rgba(0, 0, 0, 0.05) 0 4px 6px -2px;

  --sans: system-ui, 'Segoe UI', Roboto, sans-serif;
  --heading: system-ui, 'Segoe UI', Roboto, sans-serif;
  --mono: ui-monospace, Consolas, monospace;

  font: 18px/145% var(--sans);
  letter-spacing: 0.18px;
  color-scheme: light dark;
  color: var(--text);
  background: var(--bg);
  font-synthesis: none;
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;

  @media (max-width: 1024px) {
    font-size: 16px;
  }
```

- **Index.html**



```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>frontend</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

- **route/auth.js**



```
backend > routes > auth.js > router.post('/register') callback
```

```
const express = require('express')
const bcrypt = require('bcryptjs')
const jwt = require('jsonwebtoken')
const User = require('../models/User')
const auth = require('../middleware/auth')

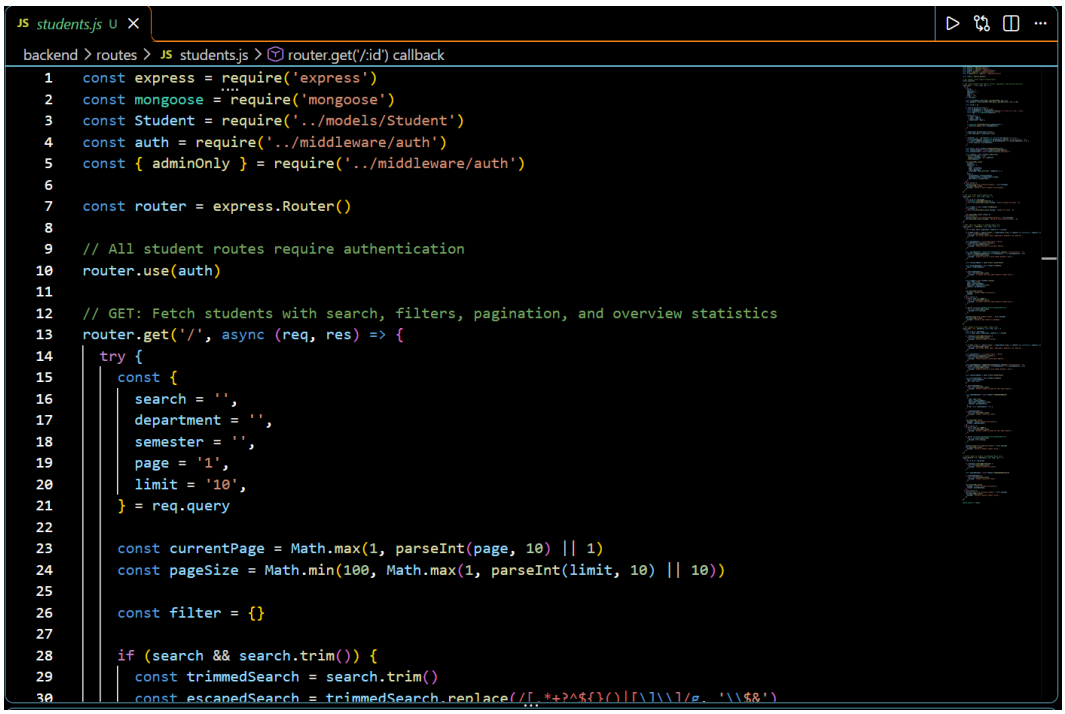
const router = express.Router()

// Helper to generate JWT token
const generateToken = (user) => {
  return jwt.sign(
    {
      userId: user._id.toString(),
      role: user.role,
      name: user.name,
      email: user.email,
    },
    process.env.JWT_SECRET || 'student-management-secret-key-default-2026',
    { expiresIn: '7d' }
  )
}

// Register a new user
router.post('/register', async (req, res) => {
  try {
    const { name, email, password, role = 'user', adminKey } = req.body

    if (!name?.trim() || !email?.trim() || !password) {
      return res.status(400).json({
        message: 'Name, email, and password are required',
```

- **route/student.js**



```
backend > routes > students.js > router.get('/:id') callback
```

```
const express = require('express')
const mongoose = require('mongoose')
const Student = require('../models/Student')
const auth = require('../middleware/auth')
const { adminOnly } = require('../middleware/auth')

const router = express.Router()

// All student routes require authentication
router.use(auth)

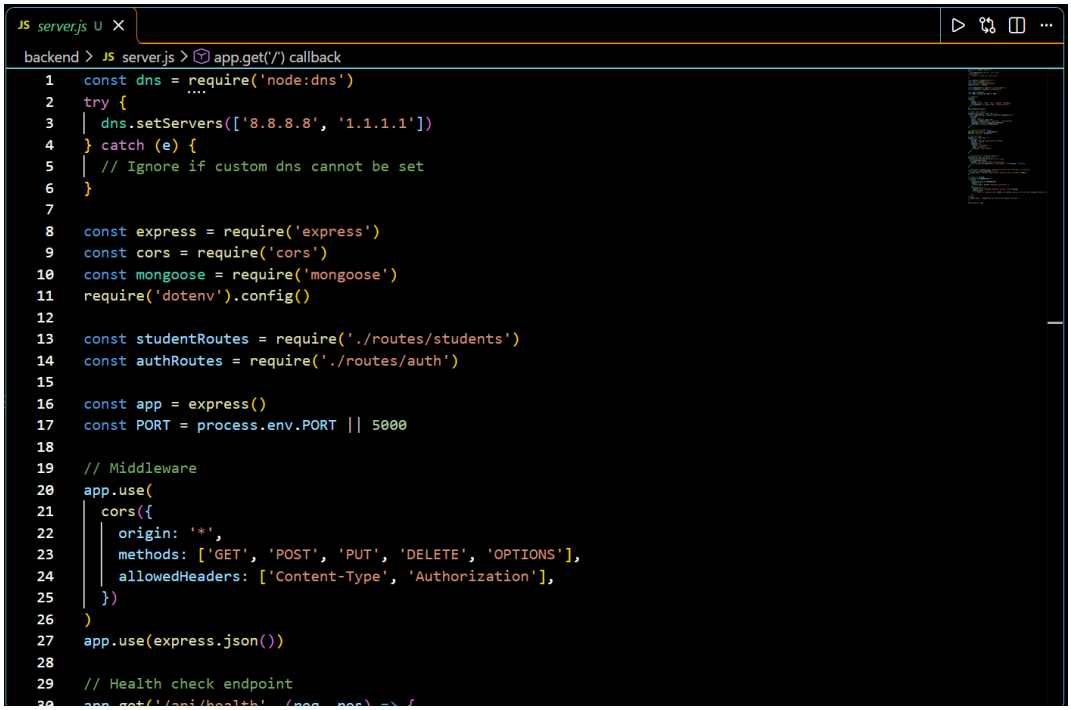
// GET: Fetch students with search, filters, pagination, and overview statistics
router.get('/', async (req, res) => {
  try {
    const {
      search = '',
      department = '',
      semester = '',
      page = '1',
      limit = '10',
    } = req.query

    const currentPage = Math.max(1, parseInt(page, 10) || 1)
    const pageSize = Math.min(100, Math.max(1, parseInt(limit, 10) || 10))

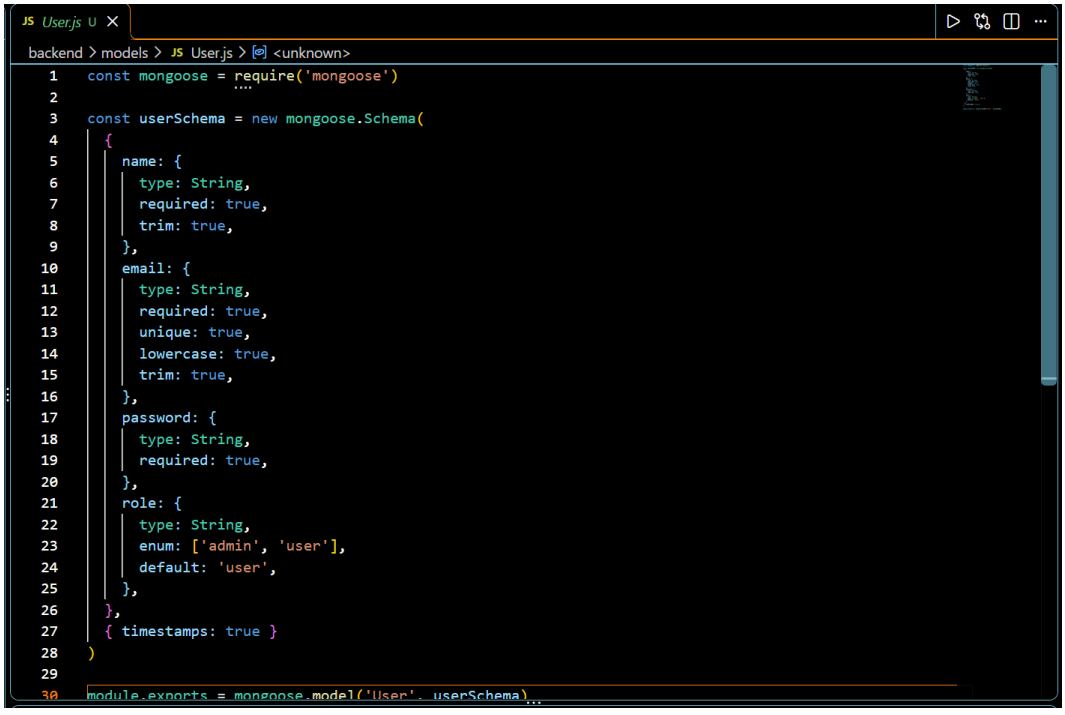
    const filter = {}

    if (search && search.trim()) {
      const trimmedSearch = search.trim()
      const escapedSearch = trimmedSearch.replace(/[.*+?^${}()|[\]\\]/g, '\\$&')
```

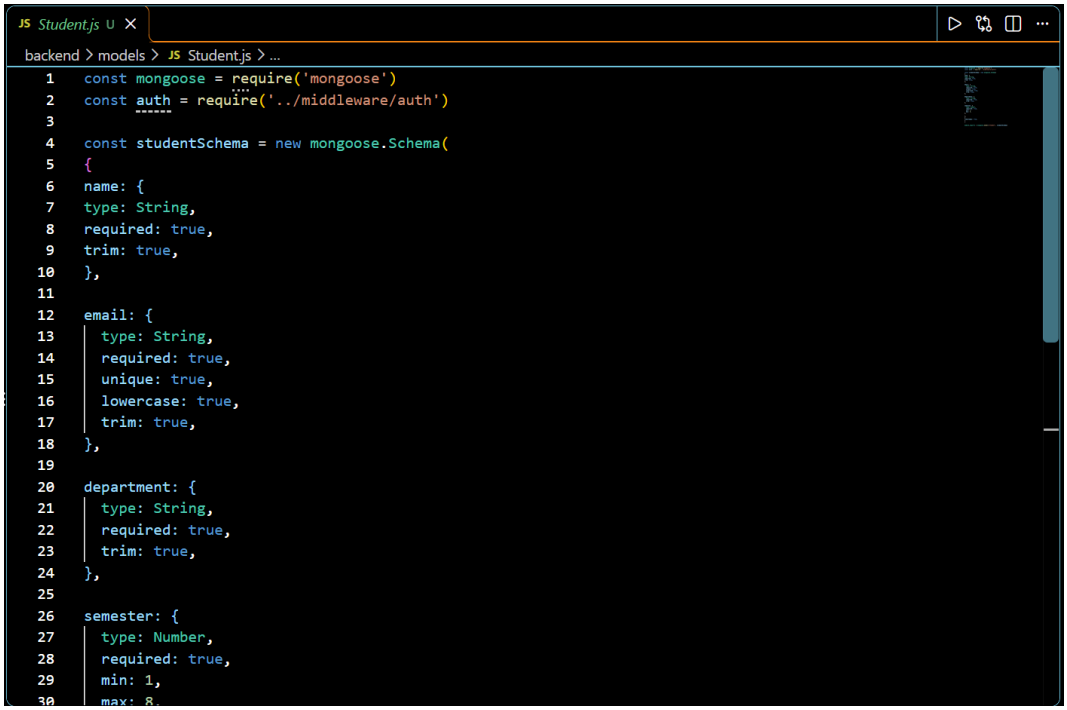
- **service.js**



- **models/user.js**



- **models/Student.js**



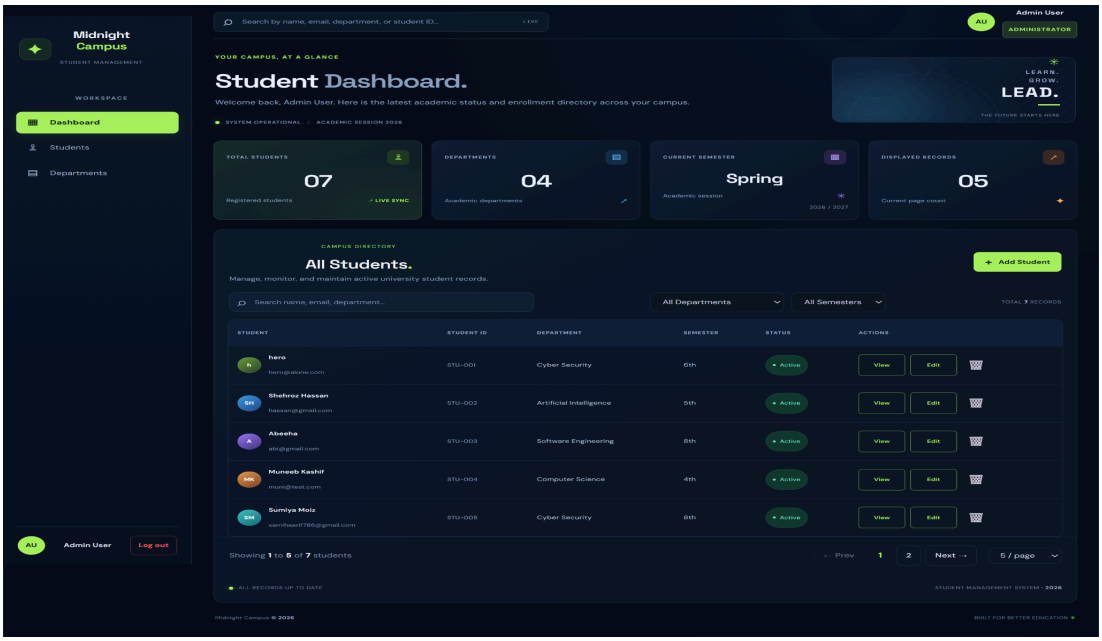
```
backend > models > JS Student.js > ...
1   const mongoose = require('mongoose')
2   const auth = require('../middleware/auth')
3
4   const studentSchema = new mongoose.Schema(
5   {
6   name: {
7   type: String,
8   required: true,
9   trim: true,
10  },
11
12  email: {
13    type: String,
14    required: true,
15    unique: true,
16    lowercase: true,
17    trim: true,
18  },
19
20  department: {
21    type: String,
22    required: true,
23    trim: true,
24  },
25
26  semester: {
27    type: Number,
28    required: true,
29    min: 1,
30    max: 8,
```

- **middleware/auth.js**

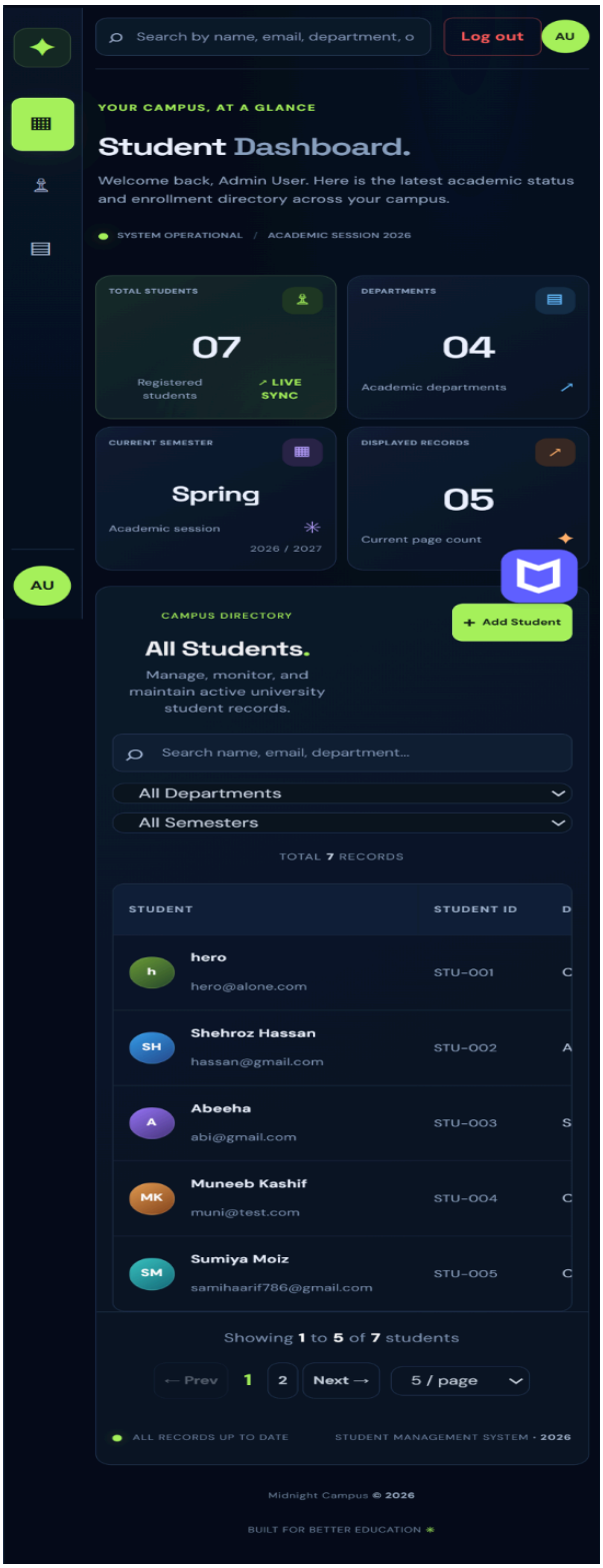


```
1   const jwt = require('jsonwebtoken')
2
3   const auth = (req, res, next) => {
4     const authHeader = req.headers.authorization
5
6     if (!authHeader || !authHeader.startsWith('Bearer ')) {
7       return res.status(401).json({
8         message: 'Access denied. Please log in.',
9       })
10    }
11
12    const token = authHeader.split(' ')[1]
13
14    if (!token) {
15      return res.status(401).json({
16        message: 'Access denied. Token missing.',
17      })
18    }
19
20    try {
21      const decoded = jwt.verify(
22        token,
23        process.env.JWT_SECRET || 'student-management-secret-key-default-2026'
24      )
25      req.user = decoded
26      next()
27    } catch (err) {
28      return res.status(401).json({
29        message: 'Invalid or expired session. Please log in again.',
30      })
```

07. Desktop Screenshots

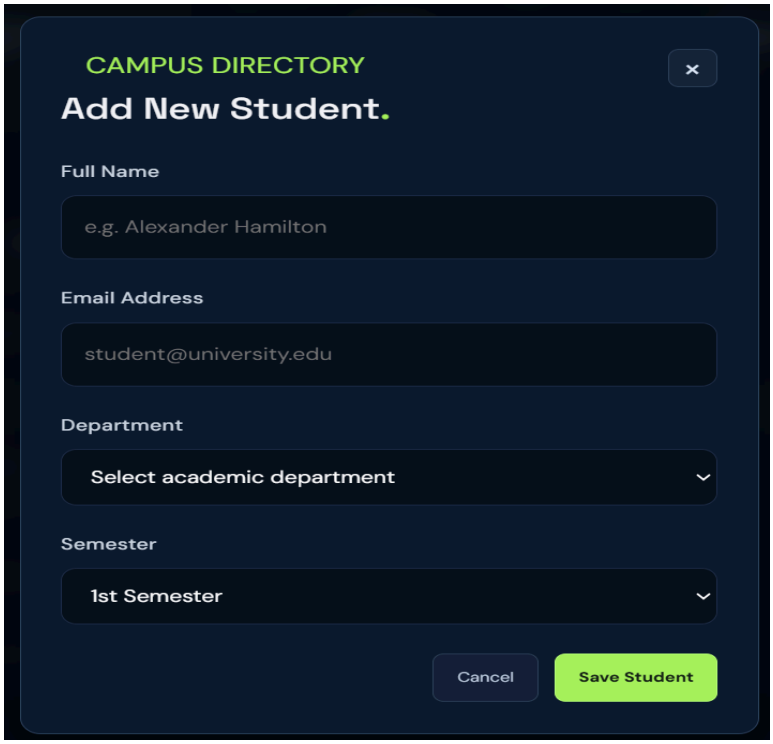


08. Mobile Screenshots

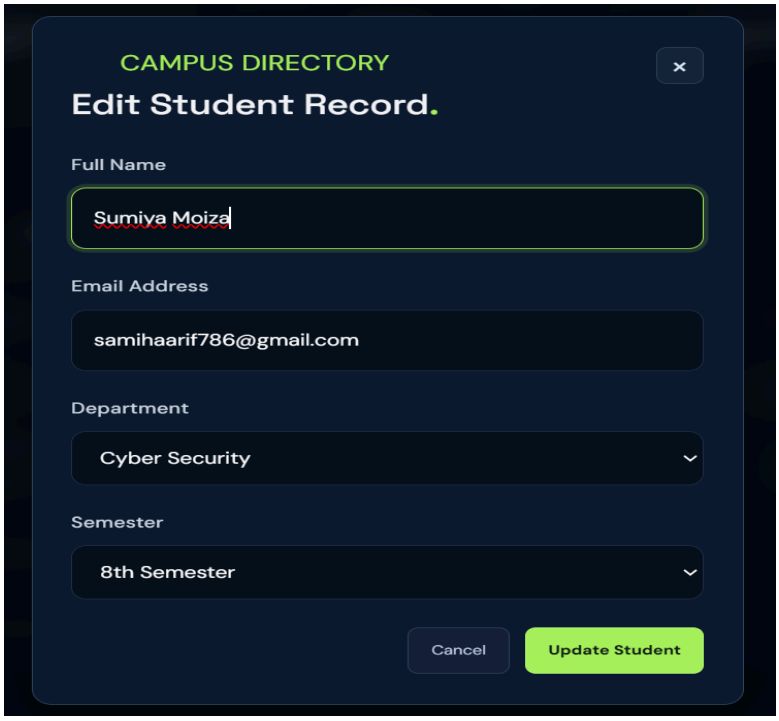


09. Project Feature Screenshots

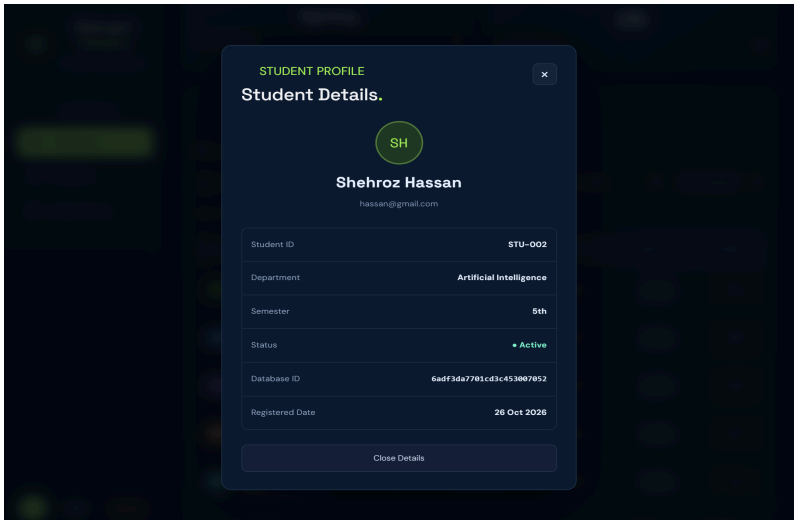
- **Add student**



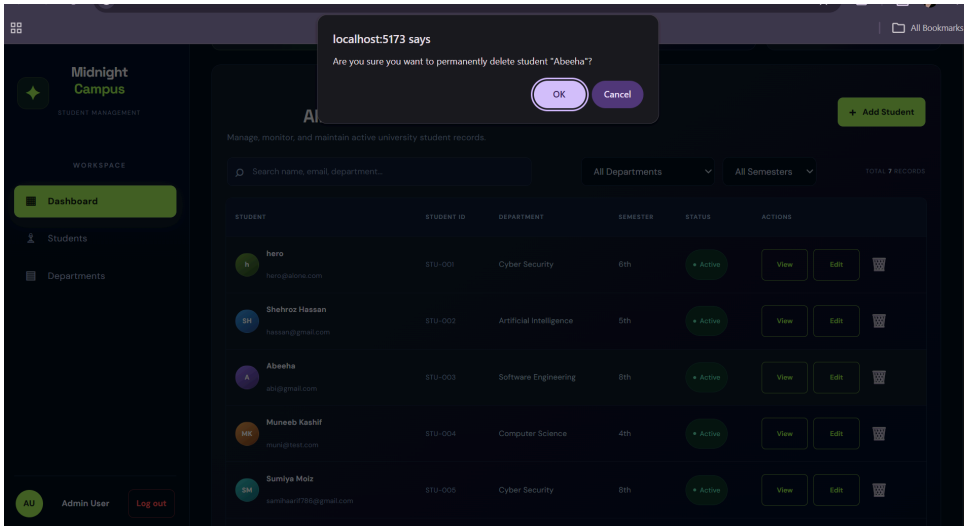
- **Edit student**



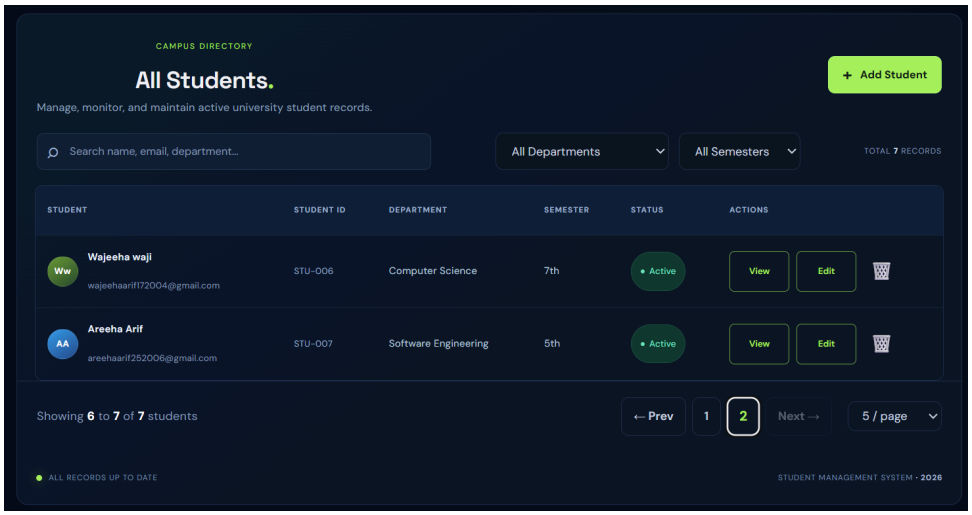
- **View student details**



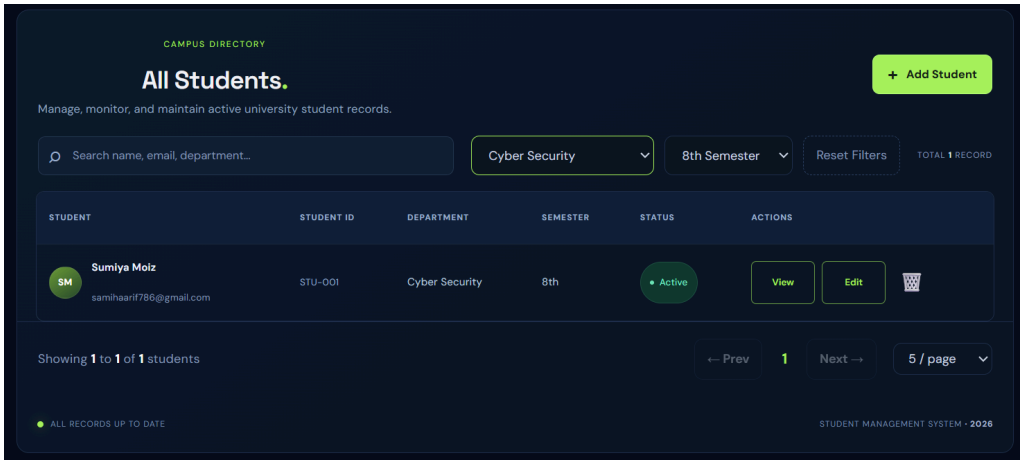
- **Delete student**



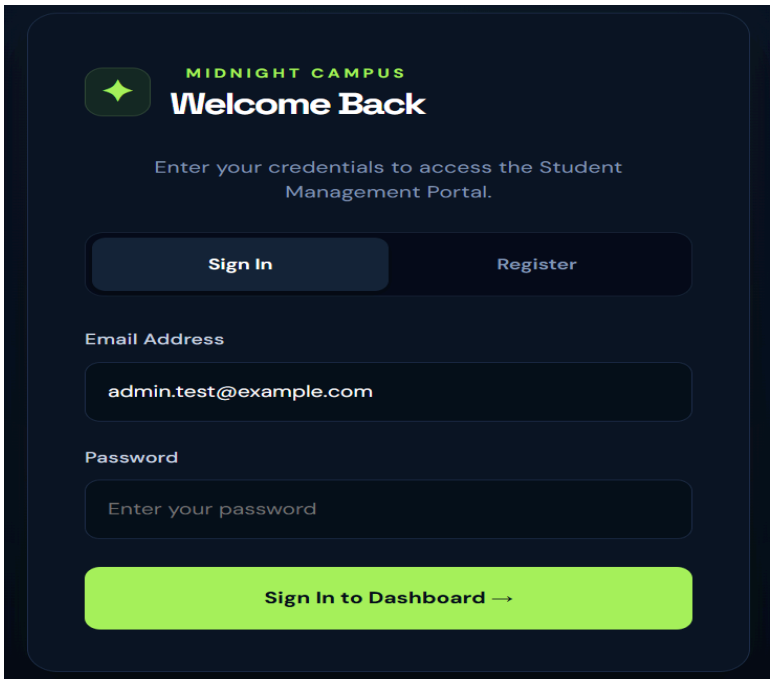
- **Pagination**



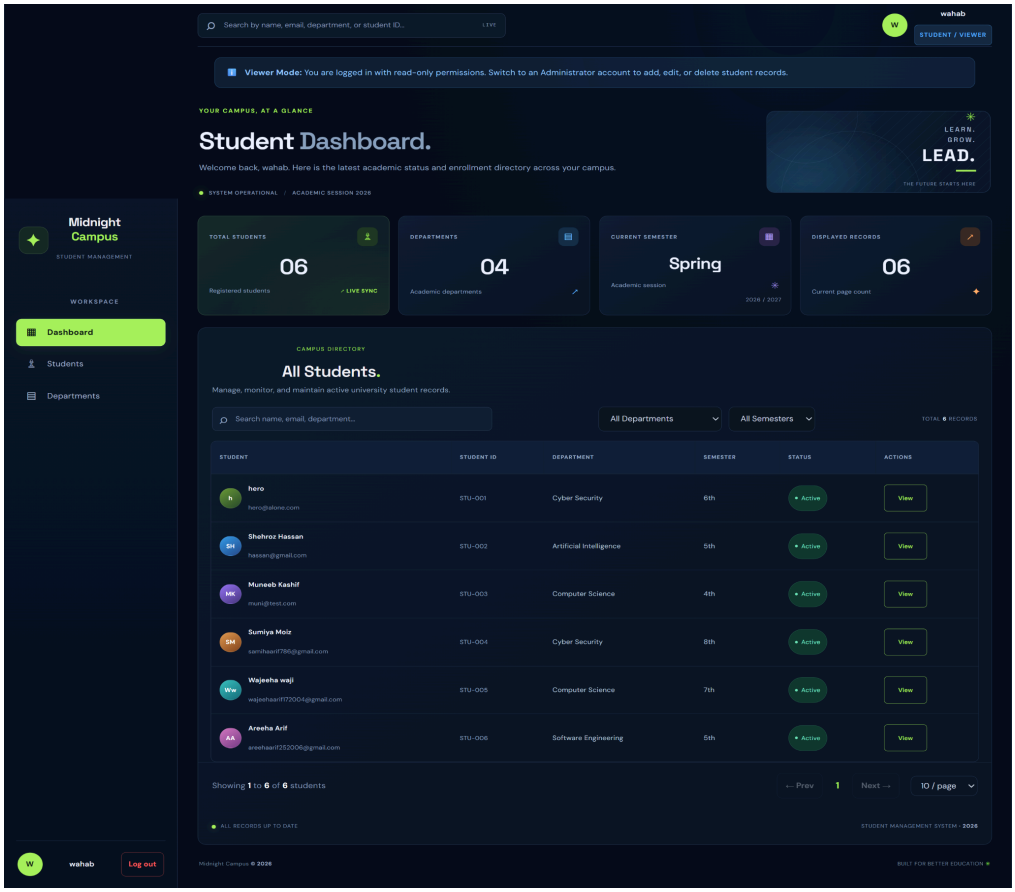
- **Filters**



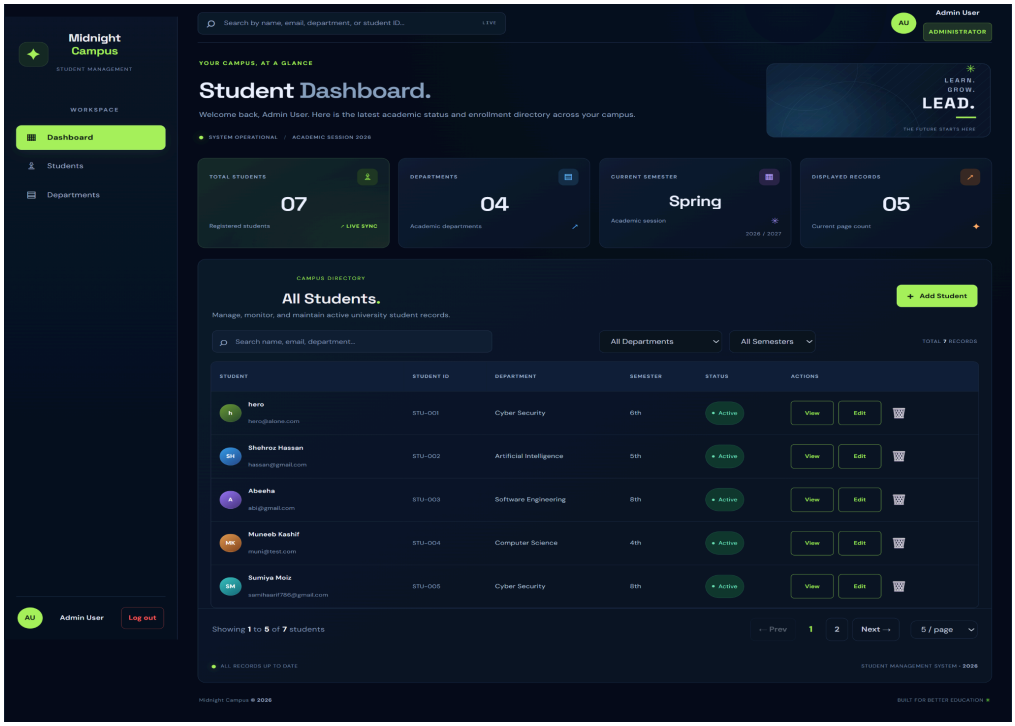
- **Authentication**



- **Viewer Dashboard**



- **Admin Dashboard**



10. Conclusion

The Midnight Campus Student Management System successfully fulfills all functional, security, and architectural criteria specified for Internship Task 3.

Key Accomplishments:

- Fully resolved the three major defects (backend connection alert, add failure, and edit null exception).
- Developed a cohesive, responsive React 19 frontend styled with custom Midnight Campus dark aesthetics.
- Deployed robust JWT authentication with role authorization and automatic session recovery.
- Implemented live multi-field search, multi-criteria filtering, and synchronized pagination.
- Established permanent, resilient cloud storage via MongoDB Atlas.

The final application represents a scalable, user-centric foundation capable of supporting university academic operations with high efficiency and data integrity.

11. GitHub Repository Link

The repository contains the source code, assets, screenshots, README, and final report.

https://github.com/arihaarif2006-web/student-management-system.git